Esquema de datos

Esta página resume los contratos principales de la integración pública de terceros.

Lectura de artículos

TipoTalleColor

```
export enum TipoTalleColor {
  NINGUNO = 0,
  TALLES = 1,
  COLORES = 2,
  TALLES_COLORES = 3
}
```

Articulo

Modelo agrupado por artículo.

```
export interface Articulo {
  articuloId: number;
  codigo: string;
  descripcion: string;
  descripcionWeb: string;
  nombre: string;
  talleColor: TipoTalleColor;

  precio1: number;
  precio2: number;
  precio3: number;
  precio4: number;
  precio5: number;

  stockTotal: number;
  curva: ArticuloCurva[];
```

```
  tags: ArticuloTag[];
}
```

ArticuloCurva

Stock por combinación de talle y color.

```
export interface ArticuloCurva {
  articuloId: number;
  colorId?: number;
  talleId?: number;
  colorNombre: string;
  colorCodigo: string;
  talleNombre: string;
  talleCodigo: string;
  unidades: number;
}
```

ArticuloConCurva

Modelo plano, útil para sincronización y procesamiento masivo.

```
export interface ArticuloConCurva {
  articuloId: number;
  codigo: string;
  descripcion: string;
  descripcionWeb: string;
  nombre: string;
  talleColor: TipoTalleColor;

  precio1: number;
  precio2: number;
  precio3: number;
  precio4: number;
  precio5: number;

  colorId?: number;
  talleId?: number;
  colorNombre: string;
  colorCodigo: string;
  colorHex: string;
  talleNombre: string;
  talleCodigo: string;
```

```
  unidades: number;
  etiquetasIds: string;
  etiquetasNombres: string;
  categoriasIds: string;
  categoriasNombres: string;
  eliminado: boolean;
}
```

RECOMENDACIÓN

Para nuevas integraciones suele ser más simple comenzar con `ArticuloConCurva`, porque cada variante queda representada como un registro independiente.

Etiquetas y categorías

TipoTag

```
export enum TipoTag {
  TODAS = -1,
  TAG = 0,
  CATEGORIA,
  MARCA,
  TEMPORADA
}
```

ArticuloTag

```
export interface ArticuloTag {
  articuloId: number;
  articuloTagId: number;
  tipo: TipoTag;
  tagId: number;
  tagNombre: string;
  padreId?: number;
  destacada: boolean;
}
```

NOTA

> En Ninox, categorías y etiquetas forman parte del mismo sistema de tags. La distinción se hace mediante el campo `tipo`.

Pedidos

CondicionIva

```
enum CondicionIva {
  SIN_CATEGORIA = 0,
  CONSUMIDOR_FINAL = 1,
  RESPONSABLE_INSCRIPTO = 2,
  MONOTRIBUTO = 3,
  EXENTO = 4,
  RESPONSABLE_NO_INSCRIPTO = 5,
}
```

PedidoTerceros

```
export interface PedidoTerceros {
  ordenId: number;
  numero: number;
  detalle: string;

  direccionEnvio: DireccionTerceros;
  direccionFacturacion: DireccionTerceros;

  productos: ArticuloTerceros[];
  usuario: UsuarioTerceros;

  subtotal: number;
  descuento: number;
  envio: number;
  recargo: number;
  total: number;
}
```

DireccionTerceros

```
export interface DireccionTerceros {
  provincia: string;
  localidad: string;
  direccion: string;
  codigoPostal: string;
}
```

UsuarioTerceros

```
export interface UsuarioTerceros {
  nombre: string;
  email: string;
  dni: string;
  cuit: string;
  telefono: string;
  condicion: CondicionIva;
}
```

ArticuloTerceros

```
export interface ArticuloTerceros {
  articuloId: number;
  precio: number;
  talleId?: number;
  colorId?: number;
  cantidad: number;
}
```

COMPORTAMIENTO OPERATIVO

Un pedido enviado por integración queda siempre dentro de la configuración del token asociado. Hoy cada app trabaja con un solo depósito y un solo punto de venta.

Respuestas

PedidoResultado

```
export interface PedidoResultado {
  facturaId: number;
  numero: number;
  pVNumero: number;
  sref: string;
  estado: number;

  datos: { [key: string]: string };

  electronica: boolean;
  cae: string;
  caevencimiento: string;
  resultado: string;
  observaciones: string;
  errorFE: string;

  saldo?: number;
  errores: boolean;
}
```

NxResultado

```
export enum NxTipoResultado {
  ERROR = 0,
  OK = 1,
  VALIDACION = 2
}

export class NxResultado {
  tipo: NxTipoResultado;
  id: number;
  data: object;
  valores: string[];
  mensajes: string[];
  errores: string[];
}
```

Recomendaciones de modelado

Si vas a construir tu propia app o middleware, lo más práctico es mapear internamente:

- una entidad de producto base

- una entidad de variante
- stock por variante
- categorías y etiquetas normalizadas
- una tabla o colección de sincronizaciones
- una tabla o colección de pedidos enviados con clave idempotente

Esto te deja preparado para la evolución del contrato a medida que se agreguen nuevos endpoints.